

计算机系统结构 (CS2305) 考试样卷

班级号：_____ 学号：_____ 姓名：_____ 成绩：_____

一、选择题 (15 分)

1. 给定一个处理器有一个七阶段的流水线，流水段分别为: F, D0, D1, I, X0, X1, W。读寄存器发生在 I 段 (issue 段)，分支是否转移要在 X0 段才确定。如果流水线是单发射的，每个时钟周期只发射一条指令，那么当执行条件转移指令转移预测错误时，需要废除多少条已经进入流水线的指令？（假设进入流水线的第一条指令是条件转移指令）（ ）

- A. 2 条
- B. 3 条
- C. 4 条
- D. 5 条

2. 关于下面程序段中的局部性 (locality)，哪些描述是正确的？（ ）

```
1 sum = 0;
2 for (i = 0; i < n; i++)
3     sum += a[i];
4 return sum;
```

- ① Sum 变量的访问呈现了时间局部性
- ② 数组元素的访问呈现了空间局部性
- ③ 循环语句的执行呈现了时间局部性
- ④ 语句的顺序执行呈现了空间局部性
- ⑤ 函数的返回语句 return 执行后，会破坏栈空间的局部性

- A. 都对
- B. 1234
- C. 123
- D. 4

3. 以下对 SIMD 的描述，哪一个是不正确的？ ()

- A. SIMD 是一种数据级并行 (data-level parallelism)，可以用一条指令同时处理多个数据。
- B. Intel 的 SIMD intrinsic 指令使用了字长比较长的寄存器，这样多个数据存放在一个寄存器中，对一个寄存器的访问等价于对多个数据的同时访问。
- C. 对控制流语句比较多的程序，例如：有比较多的转移指令、if 语句的程序，由于分支语句会对不同的数据进行不相同的操作，SIMD 的性能可能会受到影响。
- D. SIMD 是一种多线程处理器技术。

4. 哪种技术可以在执行时提升以下代码的处理速度？ ()

```
1 loop: ADD R3, R4, R0
2       LD R4, 8(R4)    # cache hit
3       BNEQZ R4, LOOP
```

- A. 带寄存器换名的乱序执行 (out of order with register renaming)
- B. 转移预测 (branch prediction)
- C. 多发射 (multiple issue)
- D. 多线程 (simultaneous multi-thread)

5. 定义了以下变量：

```
1 int x = random();
2 int y = random();
3 int z = random();
4 /* Convert to other forms */
5 unsigned ux = (unsigned)x;
6 unsigned uy = (unsigned)y;
```

从以下表达式中，选出一项不永为真 (true) 的表达式： ()

- A. $((x+y) \ll 2) + y - x == 5*y + 3*x$
- B. $x + y == ux + uy$
- C. $ux - uy == -(y - x)$
- D. $(x > y) == (-x > -y)$

6. 如果某计算机最复杂指令的执行需要完成 5 个子功能，分别由功能部件 A-E 实现，各功能部件所需时间分别为 200ps、100ps、150ps、180ps 和 100ps。采用流水线方式执行指令，流水段寄存器延时为 20ps，则 CPU 时钟周期至少为： ()

- A. 220ps
- B. 170ps
- C. 200ps
- D. 100ps

7. 以下关于超标量技术的叙述中，错误的是 ()

- A. 超标量技术开发的是指令级并行性
- B. 超标量方式执行指令时，可以同时发射多条指令至流水线中
- C. 采用超标量技术的 CPU 必须配置多个不同的功能部件
- D. 超标量技术通过提高时钟主频，加深流水线的段数，以提高指令吞吐率

8. 关于高速缓存，以下的说法哪个不正确? ()

- (a) 相同容量的全相联映射的高速缓存 (Fully associative caches) 比直接映射高速缓存访问延迟短。
- (b) 相同容量的全相联映射的高速缓存缺失率比直接映射高速缓存 (direct-mapped caches) 缺失率 (miss rate) 低。
- (c) 相同容量的组相联映射 (set associative caches) 高速缓存在缺失率上比直接映射高速缓存低。
- (d) 相同容量的组相联映射 (set associative caches) 高速缓存在访问延迟方面比全相联映射高速缓存低。

9. 下面一个 16 位的浮点数，其中 1 位符号位 (sign), 5 位阶码 (exponent), 10 位尾数位 (fraction), 下列编码哪个表示“负无穷大”? ()

- A. 1 00000 1111111111
- B. 0 11111 1111111111
- C. 0 10000 0000000000
- D. 1 11111 0000000000

10. 以下是关于控制冒险的叙述:

- 1. 直接转移指令 (无条件转移指令) 不会导致发生控制冒险
- 2. 在分支转移指令后加入若干空操作指令可避免控制冒险
- 3. 采用转发 (旁路) 技术可以解决部分控制冒险
- 4. 通过编译器调整指令顺序, 可解决一部分控制冒险, 但不能解决所有控制冒险

5. 流水线段数的深度与控制冒险引发的开销无关

以上叙述中，正确的是 ()。

A. 1, 3, 5

B. 1, 3

C. 2, 3, 4

D. 2, 4

E. 全部

二、简答题 (45 分)

1. 以 IEEE 754 单精度浮点数格式表示十进制数 -1.25，以十六进制的形式表达。(5 分)

2. 什么是精确中断？如果有一条指令在执行阶段发生了两种类型的异常：指令地址（取指）异常和 ALU 溢出，那么哪一个异常会作为指令中断原因记录？提示：以五阶段流水线处理器作为流水线模型进行回答。(5 分)

3. 什么是假共享 (false sharing)？什么时候会发生假共享？(5 分)

4. 某计算机的主存地址空间大小为 64KB，按字节编址。高速缓存 (Cache) 采用 4 路组相联映射、LRU 替换算法和写回 (write-back) 策略，能存放 4KB 数据。主存与缓存之间交换的主存块大小为 64B。请问：(1) 按字节编址，主存地址有多少位？如何划分？要求说明每个字段的含义、位数。(2 分) (2) Cache 的总容量有多少位？(3 分)

5. 简单描述一下快表 TLB (translation lookaside buffer) 的作用是什么？

6. 一个程序中不可并行化部分为 25%，可并行化部分占比 75%；如果将处理器个数从 1 个增加到 5 个，单个处理器的计算性能不变，程序可以获得的加速比是多少？

7. 假定机器 M 的时钟频率为 1GHz，某程序 P 在机器 M 上的执行时间为 12 秒钟。对 P 优化时，将其所有的乘以 2 的指令都换成了一条左移 1 位的指令，得到优化后的程序 P'。已知在 M 上乘法指令的 CPI 为 4，左移指令的 CPI 为 2，P' 的执行时间是 P 执行时间的 1.2 倍（注：原卷此处表述可能为 P 是 P' 的 1.2 倍，请按题意理解，通常优化后时间变短），则 P 中有多少条乘法指令被替换成了左移指令被执行？

8. 设一个变量的值为 4097，要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量（结果用十六进制表示），并说明哪段二进制序列在两种表示中完全相同，为什么会相同？

9. 假定一个虚拟存储系统的虚拟地址为 40 位，物理地址为 36 位，页大小为 16KB，按字节编址。若页表中有有效位、存储保护位、修改位、使用位，共占 4 位，磁盘地址不在页表中，则该存储系统中每个进程的页表大小为多少？如果按计算出来的实际大小构建页表，则会出现什么问题？

三、综合分析题 (40 分)

1. (10 分) 现在有一台机器, 它的高速缓存容量为 1024-byte, 采用直接映射, 高速缓存数据块的大小为 16-byte。给定以下定义:

```
1 struct algae_position {
2     int x;
3     int y;
4 };
5 struct algae_position grid[16][16];
6 int total_x = 0;
7 int total_y = 0;
8 int i, j;
```

并假设: (1) `sizeof(int) = 4`; (2) `grid` 数组的内存起始地址为 0; (3) 高速缓存初始时空; (4) 所有的内存访问都是对 `grid` 数组的元素进行的; (5) 变量 `i`, `j`, `total_x`, `total_y` 都存放在寄存器中。

针对对以下代码回答问题:

```
1 for (i = 0; i < 16; i++) {
2     for (j = 0; j < 16; j++) {
3         total_x += grid[i][j].x;
4     }
5 }
6 for (i = 0; i < 16; i++) {
7     for (j = 0; j < 16; j++) {
8         total_y += grid[i][j].y;
9     }
10 }
```

A. 总共有多少次读内存操作?

B. 多少次读操作引发了高速缓存失效 (miss)?

C. 高速缓存的失效率 (miss rate) 是多少?

D. 针对高速缓存失效率, 上面这段代码是否有可以改进的空间? 如何改进? 改进后高速缓存失效率为多少?

2. (10 分) 下图是一个实现了 RISC-V RV32I 指令集的单周期处理器完整的数据通路。

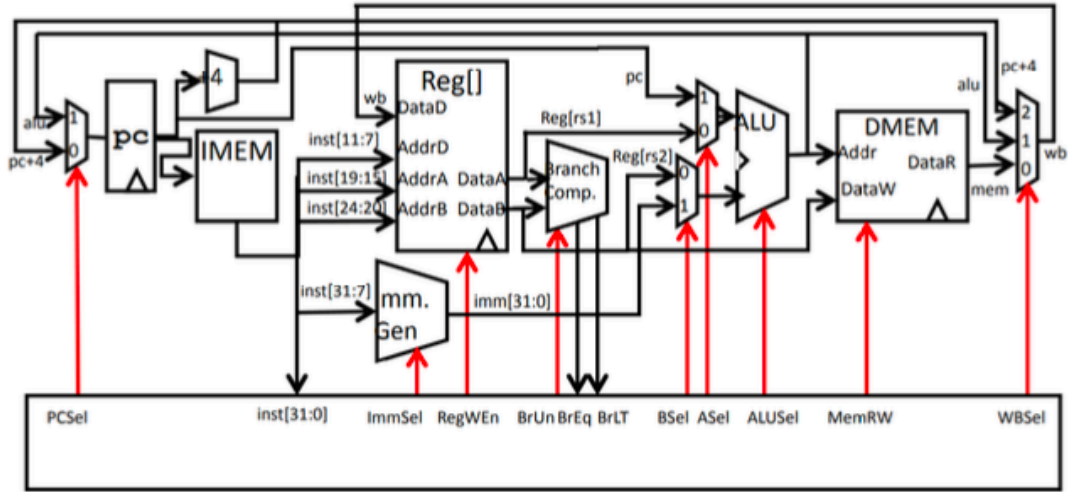


图. RV32I 单周期处理器数据通路

	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSEL	ALUSel	MemRW	RegWEn	WBSel
add	*	*	0 (PC + 4)	*	*	0 (Reg)	0 (Reg)	add	0	1	1 (ALU)
ori	*	*	0	I	*	0 (Reg)	1 (Imm)	or	0	1	1 (ALU)
lw	*	*	0	I	*	0 (Reg)	1 (Imm)	add	0	1	0 (MEM)
sw	*	*	0	S	*	0 (Reg)	1 (Imm)	add	1	0	*
beq	1/0	*	1/0	SB	*	1 (PC)	1 (Imm)	add	0	0	*
jal	*	*	1 (ALU)	UJ	*	1 (PC)	1 (Imm)	add	0	1	2 (PC + 4)
bltu	*	1/0	1/0	SB	1	1 (PC)	1 (Imm)	add	0	0	*

表. 部分指令的控制信号

图 1: RV32I 单周期处理器数据通路

表格中已经给出了部分指令应该具备的控制信号。

本题讨论 `jalr rd, rs1, imm` 指令。该指令的功能为：

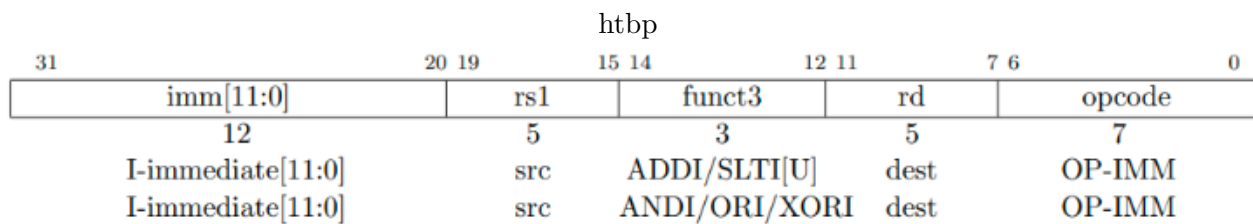
$$R[rd] = PC + 4$$

$$PC = R[rs1] + imm$$

`jalr` 属于 I 型指令，I 型指令的编码格式如下：

31	20	19 15	14 12	11 7	6	0
imm[11:0]		rs1	funct3	rd	opcode	

问题：为了支持 `jalr` 这条指令，各控制信号应该如何取值？（请列出 ImmSel, RegWEn, BrUn, BrEq, BrLT, BSEL, ASel, ALUSel, MemRW, WBSel, PCSel 等信号的值）



3. (11 分) (1) 找出下述指令之间的 RAW、WAW、WAR 数据相关性 (5 分)。

```

1 Loop: L.D    F2, 0(R1)      ; 1
2         L.D    F4, 0(R2)      ; 2
3         MUL.D  F6, F2, F4      ; 3
4         SUB.D  F4, F6, F10     ; 4
5         ADD.D  F8, F8, F4      ; 5
6         ADDI   R1, R1, -8      ; 6
7         ADDI   R2, R2, -8      ; 7
8         BNEZ   R1, LOOP        ; 8

```

(2) 接下来，我们考虑一个单发射、具有前向通路的按序流水线 (the fully-bypassed, single-issue, in-order pipeline)。如下图所示，前向通路未画出。

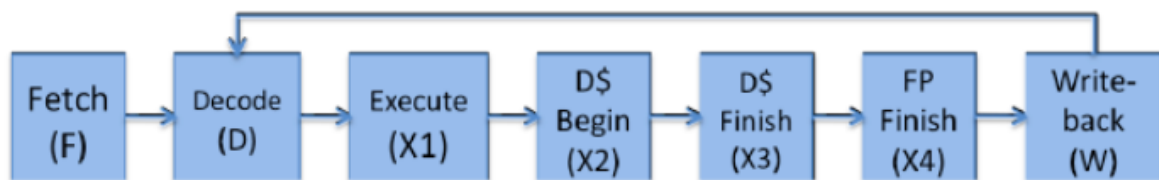


Figure 1. In-order pipeline. Full bypassing is not shown.

并有以下假设：

- 浮点加法的结果要在 X1 流水段的结尾才能获得；
- Load 指令读内存的结果要在 X3 流水段的结尾才能获得；
- 浮点乘法的结果要在 X4 流水段的结尾才能获得；
- Store 指令要写入内存的数据要在 X2 流水段时准备好即可，在 X2 流水段前不需要该数据；
- 分支指令的转移预测总是正确的（没有因为 branch 指令导致的流水线停顿）。

请画出上述八条指令一次循环的指令执行时空图。给出在不同周期时，每一条指令所处于的流水段的名称（用 F、D、X1、X2、X3、X4 和 W 表示各流水段）。(6 分)

4. (9 分) 阅读以下这段 RISC-V 代码，回答问题：

```
1 main:
2 1. addi a0, x0, 5
3 2. jal ra, FAC
4 3. addi a1, a0, 0
5 4. addi a0, x0, 1
6 5. ecall                # Print Result
7 6. addi a0, x0, 10
8 7. ecall                # Exit
9
10 FAC:
11 8. addi sp, sp, -8      # adjust stack pointer
12 9. sw ra, 4(sp)        # save ra (return address)
13 10. sw a0, 0(sp)       # save a0 (argument)
14 11. addi t0, x0, 1
15 12. bge a0, t0, L1      # if a0 >= 1, go to L1
16 13. addi a0, x0, 1      # else result = 1
17 14. addi sp, sp, 8      # adjust stack pointer
18 15. jr ra              # return to caller (1st)
19
20 L1:
21 16. addi a0, a0, -1     # decrement
22 17. jal fa              # call fa with new argument (Note: likely recursion to
    FAC)
23 18. lw t1, 0(sp)       # restore a0
24 19. lw ra, 4(sp)       # restore ra
25 20. addi sp, sp, 8      # adjust stack pointer
26 21. mul a0, a0, t1
27 22. jr ra              # return to caller (2nd)
```

1) 用 C 语言代码或者用文字描述本段代码的功能 (5 分)

2) 本段代码会输出一个整数，这个整数值为多少? (2 分)

3) 代码中第 8-10 行、第 18-20 行的作用分别是什么? (2 分)